
INPUT / OUTPUT ORGANIZATION

By: Bilal Ahmed

I/O SYSTEM DESIGN TOOL



A computer system includes three main elements: **processor** , **memory** , and **I/O modules** .



I/O modules connect peripheral devices (e.g., keyboard, disk, printer) to the system bus or central switch.



Each I/O module may control **one or more peripheral devices** .

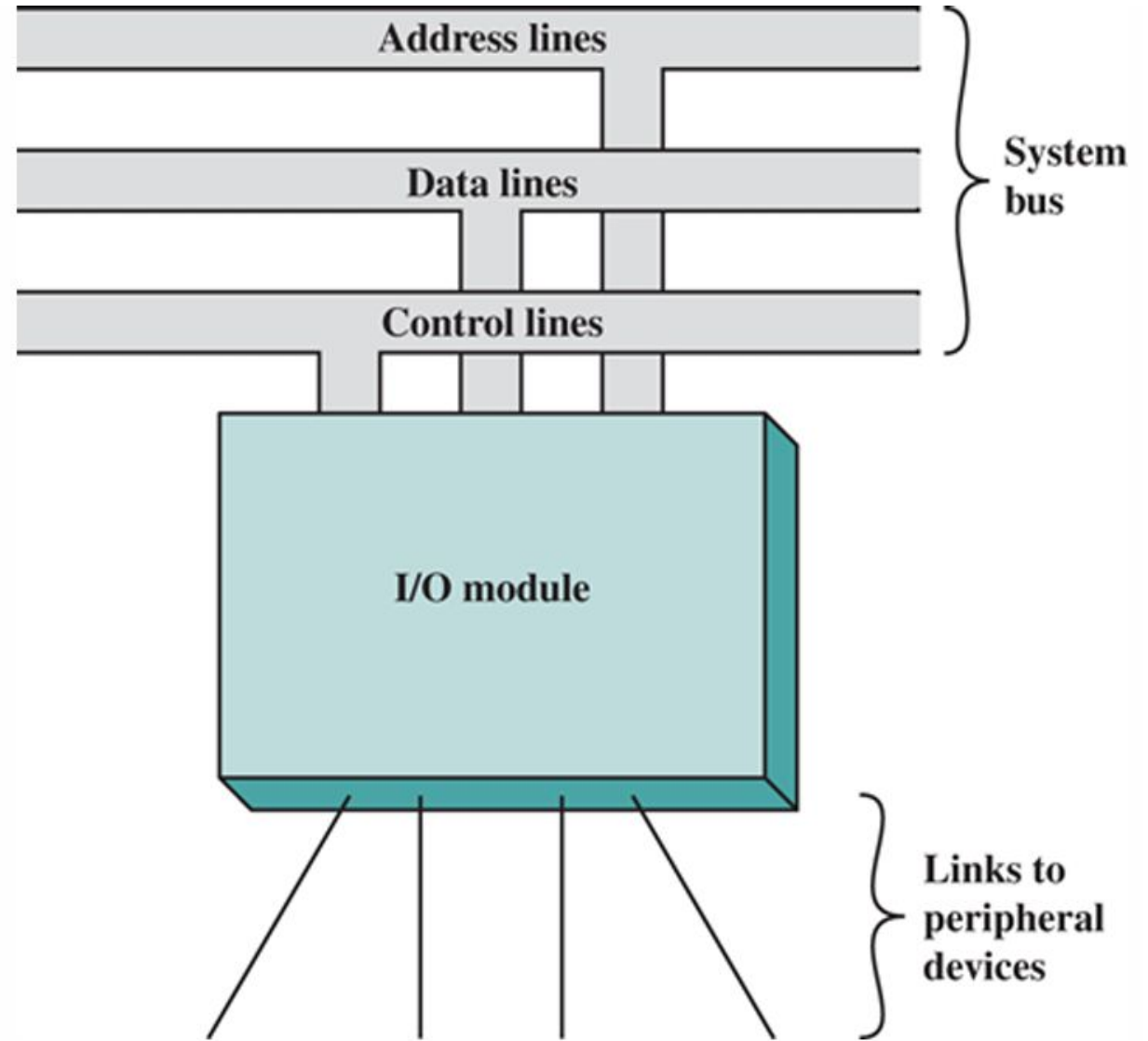


An I/O module is **not just a connector** — it includes **control logic** .



This logic manages **data transfer and communication** between the peripheral and the system bus.

I/O MODULE





EXTERNAL DEVICES

EXTERNAL DEVICES



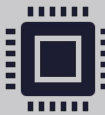
External devices enable data exchange between the **computer** and the **outside environment** .



Each external device connects to the computer via a **link** to an **I/O module** .



This link carries **control** , **status** , and **data signals** between the I/O module and the device.



Devices connected through I/O modules are called **peripheral devices** or simply **peripherals** .

EXTERNAL DEVICES TYPES

Human readable:
Suitable for
communicating
with the computer
user.

**Machine
readable:** Suitable
for
communicating
with equipment.

Communication:
Suitable for
communicating
with remote
devices.

EXTERNAL DEVICES EXAMPLES



Human-readable devices: Examples include **video display terminals (VDTs)** and **printers** .



Machine-readable devices: Include **magnetic disks** , **tape systems** , **sensors** , and **actuators** (e.g., used in robotics).



Functional view: Disks and tapes act as part of the **memory hierarchy** (data storage).



Structural view: These devices are managed by **I/O modules** , so they are treated as **I/O devices** .



Communication devices: Enable data exchange with **remote systems** , which can be human-readable, machine-readable, or **another computer** .

COMMUNICATION DEVICES

-  **Network Interface Card (NIC)** — Connects a computer to a local area network (LAN).
-  **Modem** — Converts digital signals to analog (and vice versa) for communication over telephone lines.
-  **Router** — Forwards data packets between computer networks (connects LANs/WANs).
-  **Switch** — Connects multiple devices within a network and manages data traffic efficiently.
-  **Wi-Fi Adapter** — Enables wireless communication between computers and access points.
-  **Bluetooth Module** — Provides short-range wireless communication between devices.
-  **Satellite Transceiver** — Used for long-distance communication via satellites.
-  **Serial / USB Communication Interface** — Connects peripherals or embedded systems for data exchange.



STRUCTURE AND SIGNALS OF AN EXTERNAL DEVICE INTERFACE



An external device connects to an **I/O module** through **control**, **data**, and **status** signals.



Control signals specify the operation to perform, such as:

INPUT/READ – send data to the I/O module.

OUTPUT/WRITE – receive data from the I/O module.

Status reporting or device-specific control (e.g., positioning a disk head).

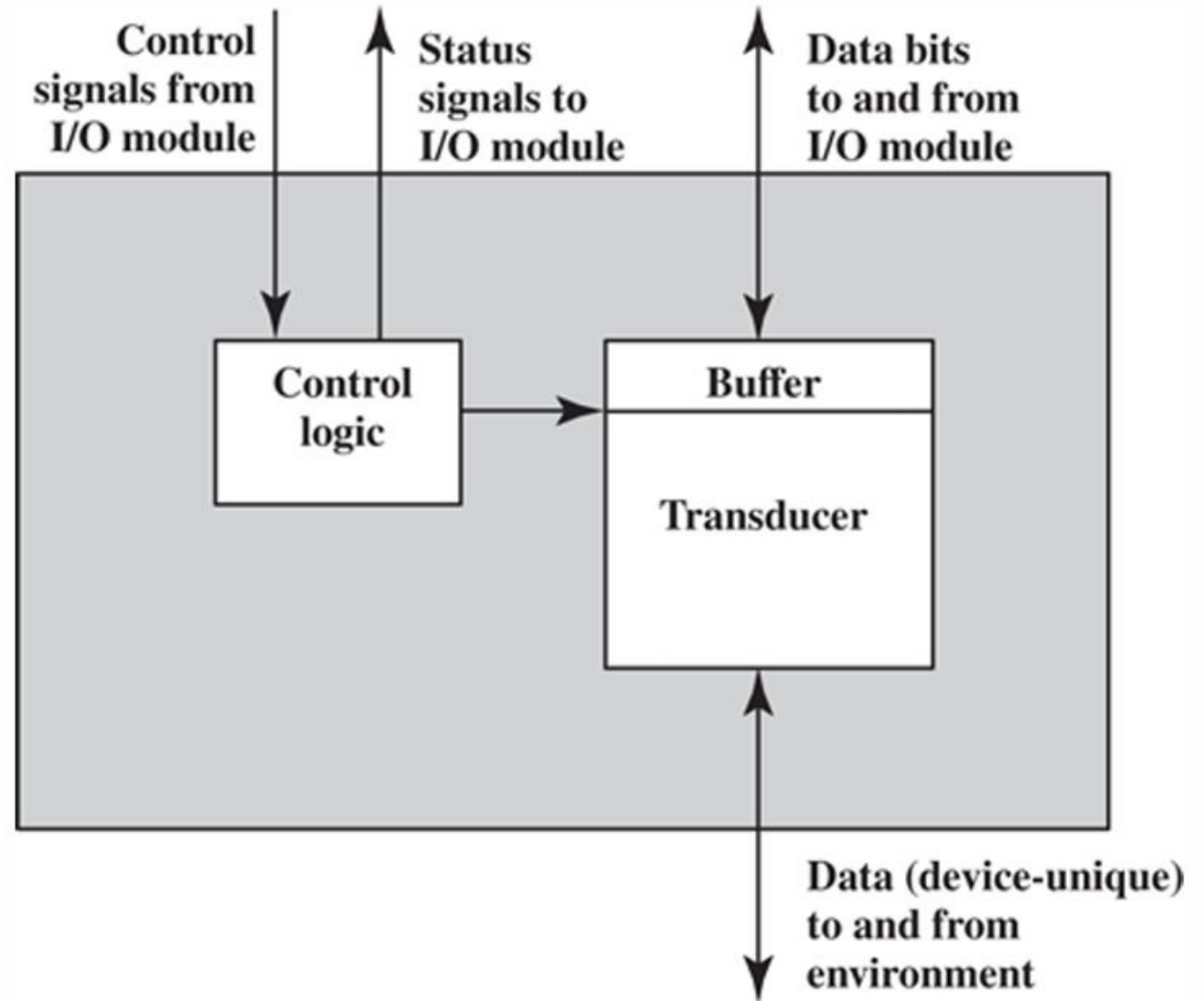


Data signals carry actual bits transferred between the device and the I/O module.



Status signals show the current state of the device (e.g., **READY/NOT-READY**).

STRUCTURE AND SIGNALS OF AN EXTERNAL DEVICE INTERFACE



OPERATION AND COMPONENTS OF AN EXTERNAL DEVICE



Control logic manages the device's operation based on commands from the **I/O module**.



The **transducer** converts:

Electrical → **physical signals** during output.
Physical → **electrical signals** during input.



A **buffer** temporarily stores data exchanged between the I/O module and the external environment.



Buffer size:

8–16 bits for serial devices.

Much larger for block-oriented devices like **disk controllers**.

KEYBOARD AND MONITOR INTERACTION



The **keyboard/monitor** setup is the most common form of **human–computer interaction** .



The **keyboard** sends input to the computer, and the **monitor** displays both user input and computer output.



Characters are the basic unit of exchange, each represented by a **7- or 8-bit code** .



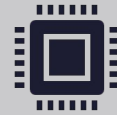
The most common encoding standard is the **International Reference Alphabet (IRA)**, also known as **ASCII** in the U.S. version.

KEYBOARD AND MONITOR INTERACTION

DISK DRIVE INTERACTION WITH I/O MODULE



A **disk drive** includes electronics to handle **data, control, and status** communication with the **I/O module**.



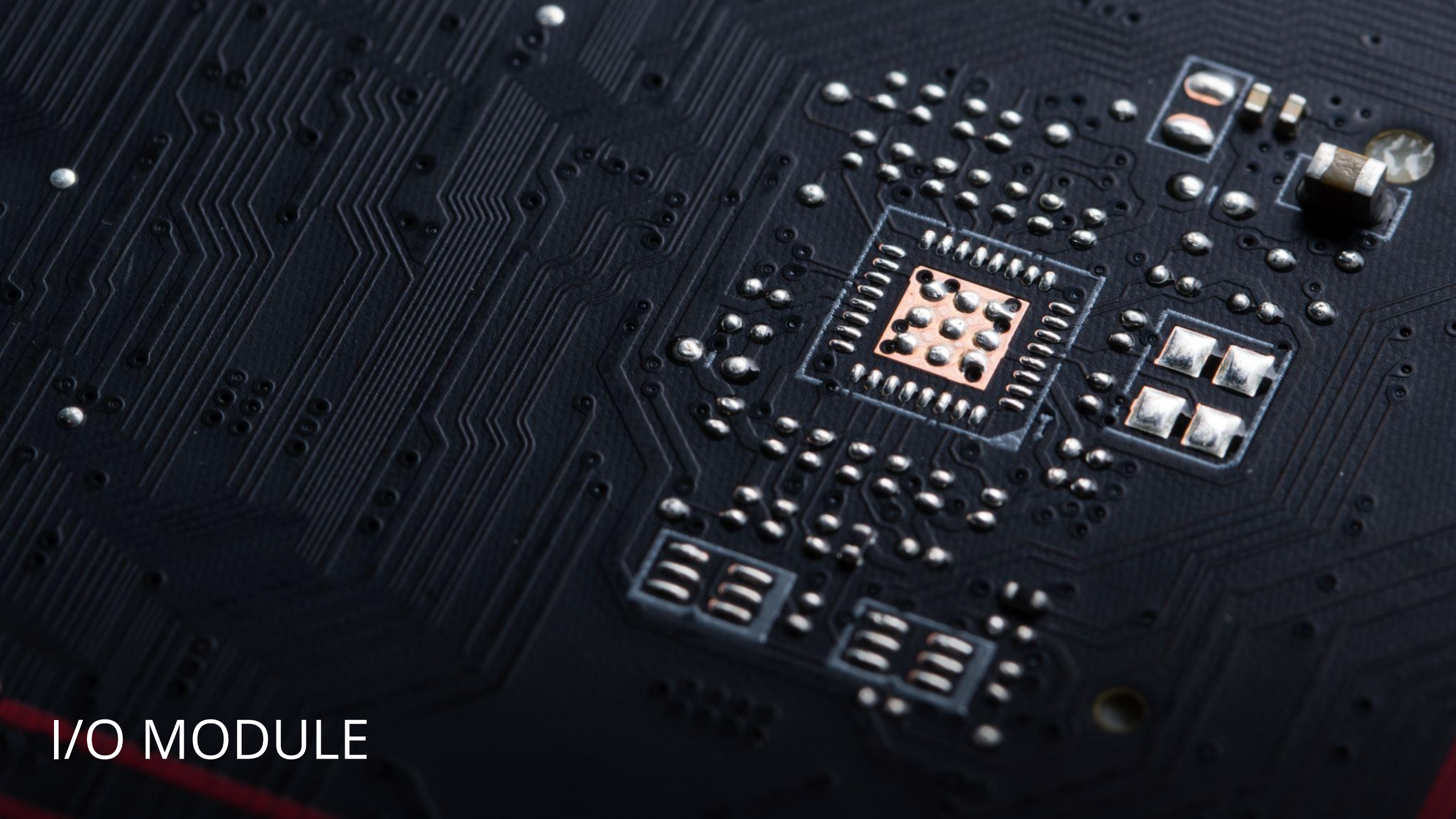
It also contains circuitry to **control** the disk's **read/write mechanism**.



In a **fixed-head disk**, the **transducer** converts **magnetic patterns** on the disk into **binary data** and vice versa.



In a **moving-head disk**, the **disk arm** moves **radially** across the surface to access different tracks.



I/O MODULE

I/O MODULES FUNCTIONS



Control and Timing: Coordinates the flow of data between processor, memory, and I/O devices.



Processor Communication: Exchanges commands, status, and data with the CPU.



Device Communication: Sends control signals and receives data/status from peripheral devices.

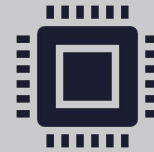


Data Buffering: Temporarily stores data to match speed differences between devices and the system bus.



Error Detection: Detects and reports transmission or device errors to ensure reliable operation.

CONTROL AND TIMING IN I/O MODULES



The **processor communicates with external devices** in varying patterns based on program needs.



System resources like memory and the system bus are **shared** among I/O and other operations.



The **I/O module manages control and timing** to coordinate data flow between devices and internal resources.

DATA TRANSFER SEQUENCE



Processor checks device status via the I/O module.



I/O module returns device status.



Processor issues a data transfer command if the device is ready.



I/O module retrieves data from the device.



I/O module transfers the data to the processor.

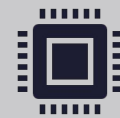
PROCESSOR COMMUNICATIO N WITH THE I/O MODULE



Command Decoding: The I/O module interprets commands from the processor (e.g., *READ*, *WRITE*, *SEEK*), often received via the **control bus** .



Data Exchange: Transfers actual data between the **processor** and **I/O module** through the **data bus** .



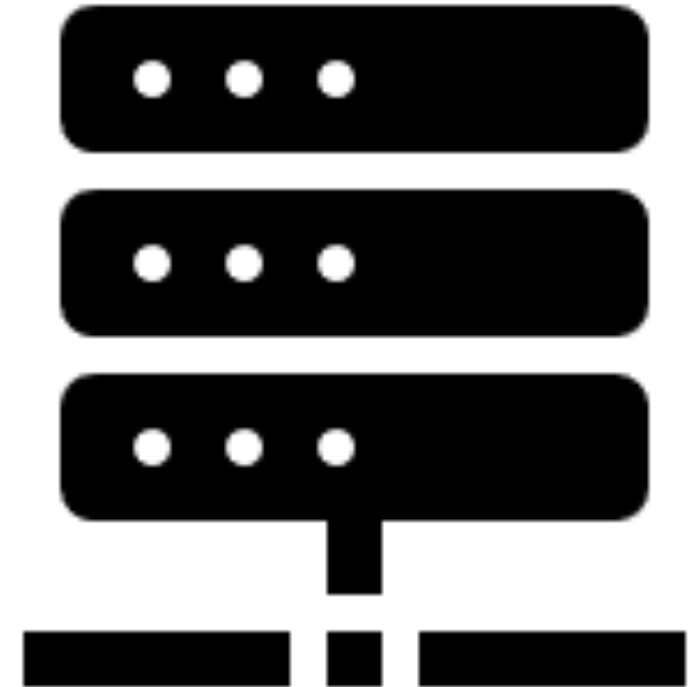
Status Reporting: Provides the processor with signals like **BUSY**, **READY**, or **ERROR** to indicate the device's current state.



Address Recognition: Each I/O module responds to a **unique address** assigned to the device it controls.

DATA BUFFERING IN I/O MODULES

- **Data buffering** is a key function of an **I/O module** to handle speed differences between devices and memory.
- **Main memory and processor** transfer data much faster than most **peripheral devices**.
- The I/O module **stores data temporarily (buffering)** to match these different transfer rates.
- During output, data are received quickly from memory and released slowly to the device.
- During input, data are collected from the device and held until memory is ready to receive them.



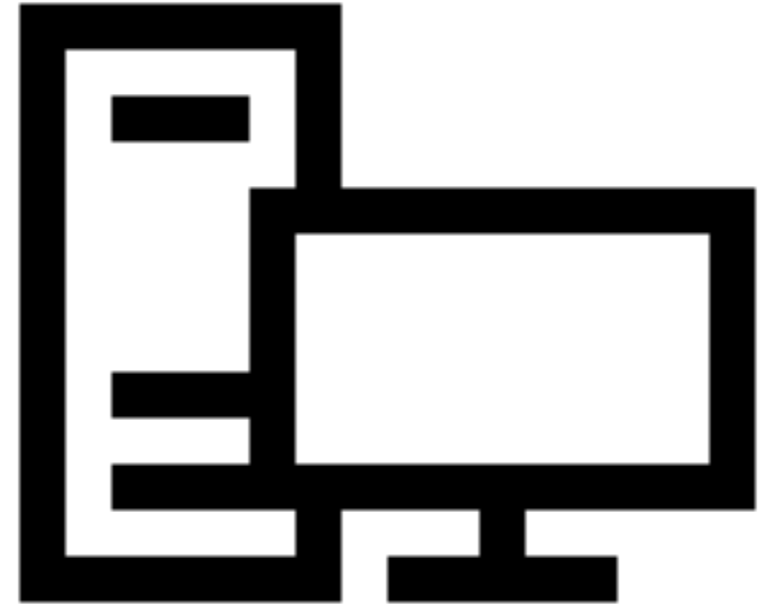
ERROR DETECTION IN I/O MODULES

- The **I/O module** detects and reports **errors** to the **processor**.
- Mechanical/Electrical errors:** Issues like **paper jams** or **bad disk tracks** are reported by the device.
- Transmission errors:** Occur when **bit patterns change** during data transfer between the device and I/O module.
- Error-detecting codes** (e.g., **parity bits**) are used to identify such errors.
- In **parity checking**, the 8th bit ensures the total number of 1s is **even** or **odd**, allowing the I/O module to verify data integrity on receipt.

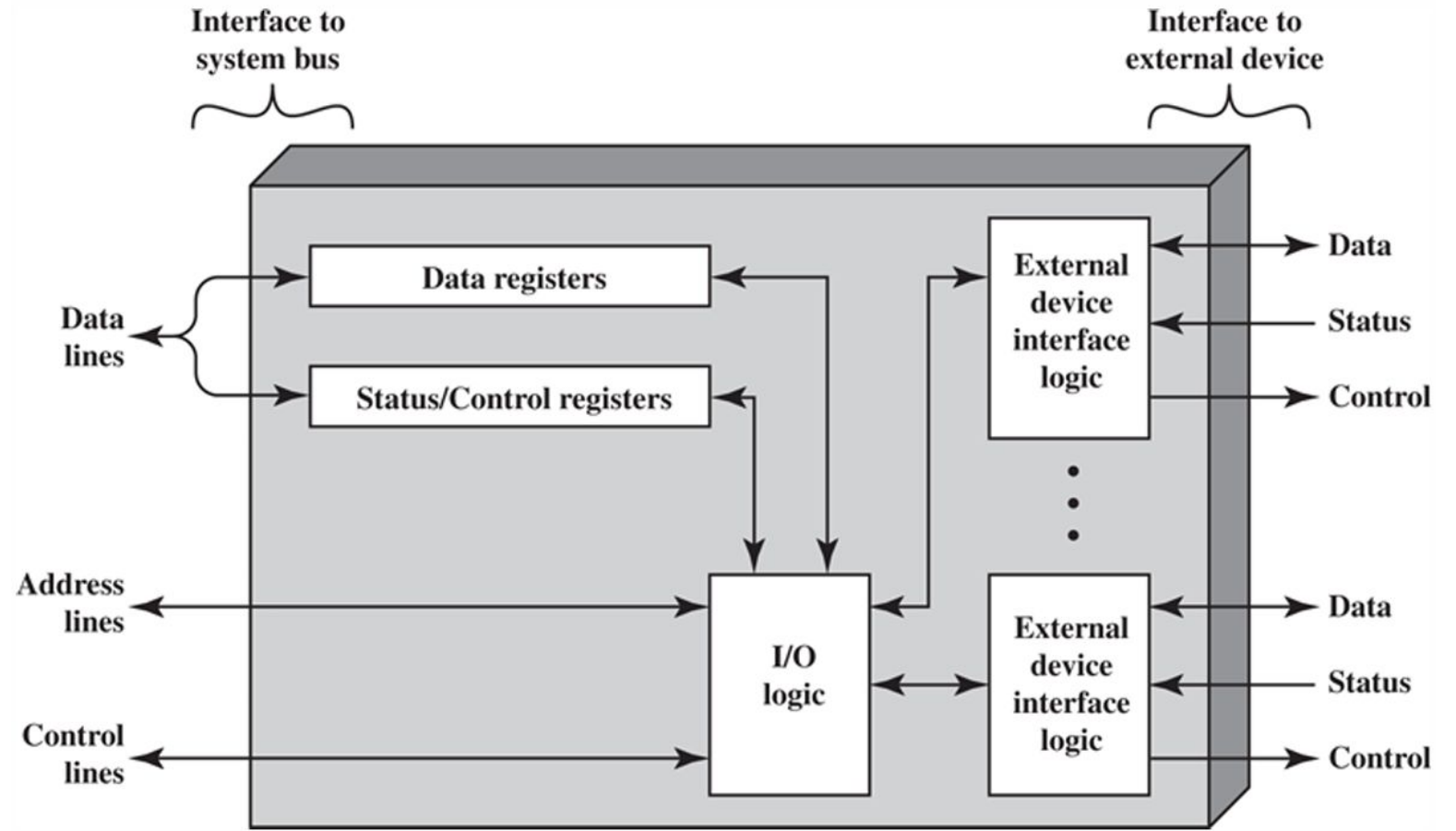


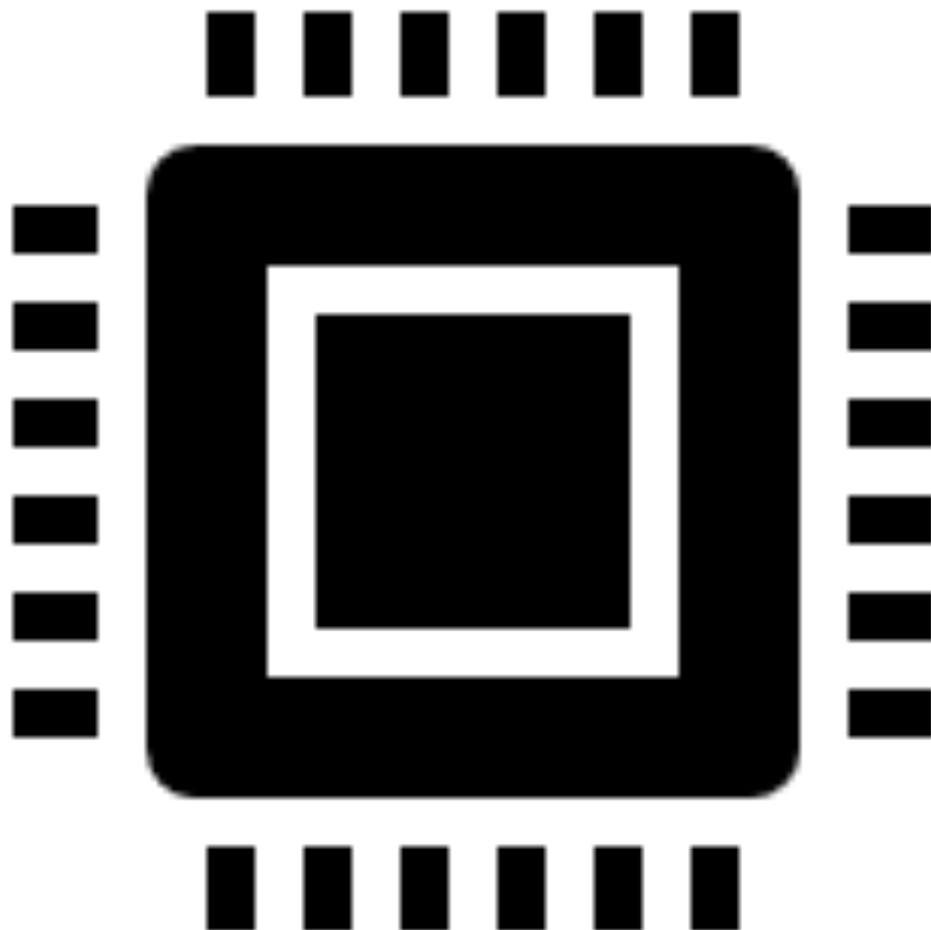
STRUCTURE OF AN I/O MODULE

- **I/O modules** differ in complexity and in the number of **devices** they manage.
- The module connects to the **system bus** through a set of **signal lines** for data, address, and control.
- **Data registers** buffer information transferred between the processor and devices.
- **Status/control registers** store device status and receive control commands from the processor.
- **Control logic** manages communication and timing via **control lines**.
- Each I/O module has a **unique address (or set of addresses)** for the devices it controls.
- The module includes **device-specific interface logic** to handle communication with individual peripherals.



STRUCTURE OF AN I/O MODULE (INTEL 8255A)

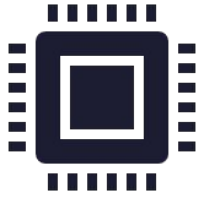




PROGRAMME

D I/O

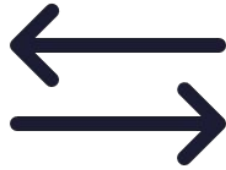
TECHNIQUES FOR I/O OPERATIONS



Programmed I/O:

The processor directly controls all I/O operations (status check, read/write, data transfer).

The CPU **waits** until the I/O is complete → leads to **processor idle time**.



Interrupt-Driven I/O:

The processor issues an I/O command and continues executing other instructions.

The **I/O module interrupts** the CPU once the operation is complete → **better efficiency**.



Direct Memory Access (DMA):

The **I/O module transfers data directly** between memory and the device.

The **processor is bypassed** during the transfer → allows **high-speed data movement** with minimal CPU overhead.

COMPARISON OF I/O TECHNIQUES

Technique	CPU Involvement	Speed	Efficiency	Best Used When
Programmed I/O	High – CPU controls every step	Slow	Low (CPU waits idly)	Simple devices or small data transfers
Interrupt-Driven I/O	Moderate – CPU only acts when interrupted	Faster than Programmed I/O	Medium to High	Moderate-speed devices (keyboard, mouse, etc.)
Direct Memory Access (DMA)	Very Low – CPU only starts and ends transfer	Fastest	Very High (CPU free for other tasks)	High-speed or large data transfers (disk, network, etc.)

PROGRAMMED I/O OPERATIONS



When an I/O instruction is executed, the **processor sends a command** to the relevant **I/O module**.



The **I/O module** performs the requested action (e.g., read or write).



Once done, it updates the **I/O status register** to indicate completion or device state.



The **I/O module does not interrupt** the processor after finishing.



Therefore, the **processor must repeatedly check (poll)** the I/O module's status register to detect when the operation is complete.

PROCESSOR VS. INSTRUCTION VIEW IN PROGRAMME D I/O

Processor View:

- The CPU **issues I/O commands** (e.g., *READ, WRITE, CONTROL, TEST*) to the I/O module.
- The I/O module executes these commands and interacts with the connected device.

Instruction View:

- The CPU **fetches and executes I/O instructions** from memory that correspond to specific I/O commands.
- Each instruction includes the **device address** and **operation type**, defined by the addressing mode (*memory-mapped* or *isolated I/O*).

I/O COMMANDS



The **processor issues an address and an I/O command** to the targeted I/O module and device.



Control: Activates the peripheral and specifies an action (e.g., rewind or move tape forward).



Test: Checks the **status** of the device — power, readiness, completion, or errors.



Read: Retrieves data from the peripheral into the I/O module's **buffer/data register** for the processor to access.



Write: Sends data from the **processor (via data bus)** to the peripheral device for output.

I/O INSTRUCTIONS

- **I/O instructions** correspond directly to **I/O commands** sent from the processor to the I/O module.
- Each **I/O device** is assigned a **unique address or identifier** for communication.
- The **I/O module** checks address lines to determine if a command is meant for it.
- Two main **addressing modes** are used:
 - **Memory-Mapped I/O:** I/O devices share the same address space as memory.
 - **Isolated I/O:** I/O devices have a **separate address space** from memory.
- Both methods allow the processor to **read/write data** to I/O devices through standard instructions.





INTERRUPT-D RIVEN I/O

INTERRUPT-DRIVEN I/O

In **programmed I/O** , the processor wastes time **polling** the I/O module to check if it's ready.

This causes **low system performance** due to continuous waiting.

Interrupt-driven I/O solves this by letting the processor **issue a command** and continue other tasks.

When the I/O module is ready, it **sends an interrupt signal** to the processor.

The processor then **pauses current work** , performs the **data transfer** , and **resumes execution** afterward.

WORKING OF INTERRUPT-DRIVEN I/O

I/O Module View:

- Receives a **READ command** from the processor.
- Reads data from the **peripheral** into its **data register**.
- Sends an **interrupt signal** to the processor once data is ready.
- Waits until the processor requests the data, then places it on the **data bus**.

Processor View:

- Issues a **READ command** and continues other work.
- Periodically checks for **interrupt signals** at the end of each instruction cycle.
- When interrupted, **saves current context**, handles the I/O interrupt, and reads data from the module.
- **Stores the received data in memory**, restores the previous context, and resumes normal execution.

INTERRUPT PROCESSING IN INTERRUPT-DR IVEN I/O



Interrupt Signal: The device sends an **interrupt request** to the processor after completing an I/O operation.



Instruction Completion: The processor **finishes its current instruction** before responding to the interrupt.



Acknowledge Interrupt: The processor detects the interrupt, **sends acknowledgment** to the device, allowing it to clear the request.



Save Context: The processor **stores the current state** — Program Counter (PC) and Program Status Word (PSW) — onto the **system stack** to resume later.

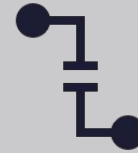


Load Interrupt Handler: The **Program Counter** is updated with the **address of the interrupt service routine (ISR)**.



Save Registers: The ISR **pushes all processor registers** onto the stack to preserve the current program state.

DESIGN ISSUES IN INTERRUPT I/O

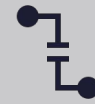


Identifying the Source: Since multiple I/O modules can request interrupts, the processor must determine **which device** issued the interrupt signal.



Handling Multiple Interrupts: When several interrupts occur at once, the processor must decide **which interrupt to handle first**, often based on **priority levels** or predefined interrupt-handling schemes.

DEVICE IDENTIFICATION TECHNIQUES IN INTERRUPT I/O



Multiple Interrupt Lines – Each device has a dedicated interrupt line to the processor.



Software Poll – The processor checks each I/O module sequentially to find which one raised the interrupt.



Daisy Chain (Hardware Poll / Vectored) – Devices are connected in a chain; the priority is determined by their physical order.



Bus Arbitration (Vectored) – Devices share the bus and use a priority-based arbitration mechanism to identify and handle interrupts efficiently.



DIRECT MEMORY ACCESS

DRAWBACKS OF PROGRAMMED AND INTERRUPT-DRIVEN I/O

Both techniques require **processor involvement** in every data transfer.

The **I/O transfer rate** is limited by how fast the processor can test and service devices.

The **processor is occupied** executing multiple instructions for each I/O transfer.

Programmed I/O: Faster data movement but fully ties up the processor.

Interrupt-Driven I/O: Frees processor partially but offers slower transfer rates.

Result: Neither is efficient for large data volumes — **Direct Memory Access (DMA)** is preferred.

DIRECT MEMORY ACCESS (DMA) FUNCTION

DMA (Direct Memory Access) adds a dedicated module to the system bus for high-speed data transfer.

The **DMA module** can temporarily take control of the system bus — acting like a mini-processor.

It transfers data **directly between memory and I/O devices**, bypassing the CPU.

Method 1: Uses the bus when the processor is idle, or

Method 2 (Commonly used): Temporarily suspends the processor to use the bus (known as **cycle stealing**).

This method greatly improves efficiency for **large data transfers**.

DMA INITIATES DATA TRANSFER

The **CPU** issues a **DMA command** when a large data block needs to be read or written.

- CPU provides the DMA module with:
- Operation type (**Read / Write**)
- **I/O device address**
- **Starting memory location** (→ stored in DMA address register)
- **Number of words** to transfer (→ stored in data count register)

After setup, the **CPU continues other tasks** — control is temporarily passed to the DMA module.

DMA PERFORMS AND FINISHES DATA TRANSFER

The **DMA transfers data directly** between memory and the I/O device **without CPU involvement** .

Uses **cycle stealing** — briefly pauses the CPU for one bus cycle (not a full interrupt).

When the transfer completes, the **DMA sends an interrupt** to the CPU.

Faster block data transfer.

CPU is mostly free for other work.

Much **more efficient** than programmed or interrupt-driven I/O.